

We are starting at **13:00!**

Grab a seat and get ready



Saturdays.AI
Kigali

#4 Simple ML for Classification

AI Saturdays Kigali

Simple ML for Classification (Linear Classifiers)



Saturdays.AI
Kigali

Linear Classifiers in Python

In today's class, we'll explore two foundational classification algorithms in machine learning:

- **Perceptron**
- **Adaptive Linear Neuron (Adaline)**

What You'll Learn

- Step-by-step **implementation of the Perceptron** in Python
 - Apply it to classify flower species using the Iris dataset
 - Understand how basic ML algorithms work and how to code them
- Introduction to **optimization** with Adaline
 - Prepares you for using **advanced models** with **scikit-learn**

Tools & Topics Covered

- Building an understanding of machine learning algorithms
- Using **Pandas**, **NumPy**, and **Matplotlib** to read in, process, and visualize data
- Implementing **linear classifiers** for 2-class problems in Python

A Brief History of Artificial Neurons

- **1943:** McCulloch & Pitts introduced the **first artificial neuron model**, inspired by how biological neurons work
 - Their neuron acted like a **logic gate**: combining inputs and firing if a threshold was reached

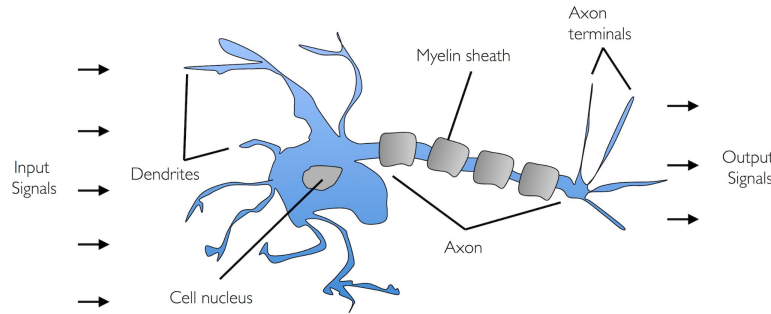


Figure 2.1: A neuron processing chemical and electrical signals

- **1957:** Frank Rosenblatt developed the **Perceptron**
 - Built on the McCulloch-Pitts (MCP) neuron
 - Introduced a learning rule to **adjust weights automatically**
 - Could be used for **binary classification** in supervised learning

These early ideas laid the foundation for modern neural networks and classification algorithms.

Formal Definition of an Artificial Neuron

We model an artificial neuron for binary classification (Class 0 or 1) using:

Net Input (Linear Combination)

$$z = w_1x_1 + w_2x_2 + \dots + w_mx_m:$$

Where:

- x = the input features
- w = the learned weights

$$\mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

Decision Function, $\sigma(\cdot)$

- Predict class 1 if $z \geq \theta$, otherwise class 0
- This is a **unit step function**:

$$\sigma(z) = \begin{cases} 1 & \text{if } z \geq \theta \\ 0 & \text{otherwise} \end{cases}$$



Formal Definition of an Artificial Neuron

Simplifying for Implementation

1. Move the threshold to the left:

$$\begin{aligned} z &\geq \theta \\ z - \theta &\geq 0 \end{aligned}$$

2. Introduce a bias term $b = -\theta$

$$z = w_1x_1 + \dots + w_mx_m + b = \mathbf{w}^T \mathbf{x} + b$$

3. Final decision function:

$$\sigma(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

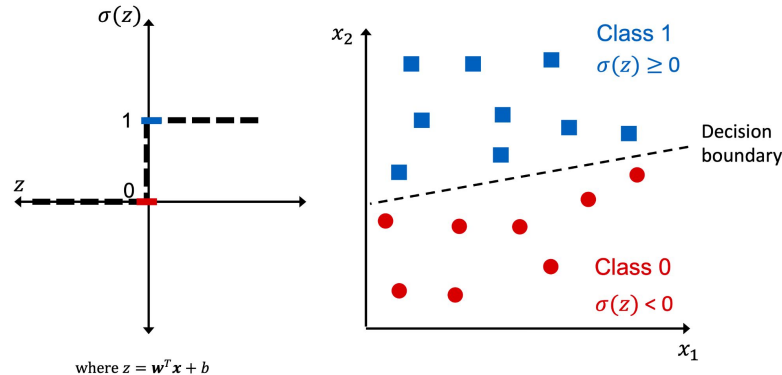


Figure 2.2: A threshold function producing a linear decision boundary for a binary classification problem

The Perception Learning Rule

The Perceptron Learning Rule

Inspired by how biological neurons fire or don't, the perceptron uses a simple threshold-based model to perform binary classification.

Algorithm Overview

1. Initialize weights and bias to **zeros (0)** or **small random values**
2. For **each training example**, $\mathbf{x}^{(i)}$:
 - a. Compute the output value, $\hat{y}^{(i)}$
 - b. Update the weights and bias unit



The Perceptron Learning Rule

Update Equations

- Weight and Bias update:

$$w_j := w_j + \Delta w_j$$

and $b := b + \Delta b$

- The update values (“deltas”):

$$\Delta w_j = \eta(y^{(i)} - \hat{y}^{(i)})x_j^{(i)}$$

and $\Delta b = \eta(y^{(i)} - \hat{y}^{(i)})$

- η = learning rate (typically a constant between 0.01 and 1.0)
- $y^{(i)}$ = true label
- $\hat{y}^{(i)}$ = predicted label



The Perceptron Learning Rule

Convergence Condition

- The perceptron **only converges** if the two classes are **linearly separable**
- If not:
 - Set a **maximum number of epochs**, or
 - Use a **threshold** for the number of tolerated misclassifications→ Otherwise, the perceptron **would never stop updating the weights**

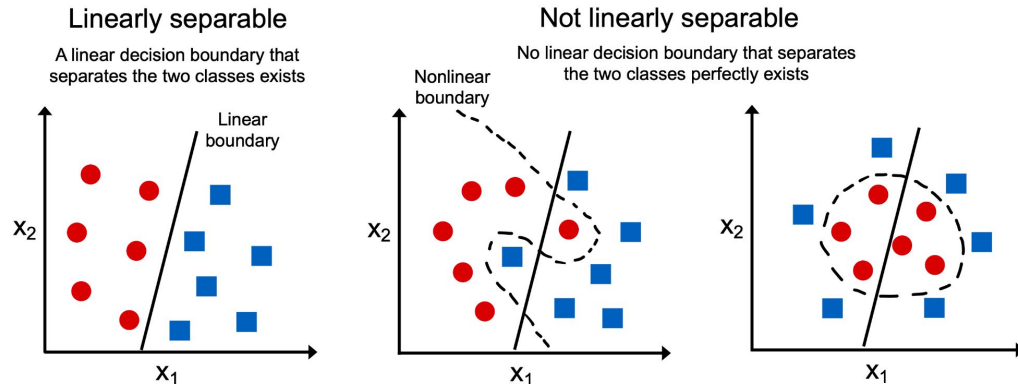


Figure 2.3: Examples of linearly and nonlinearly separable classes

The Perceptron Learning Rule

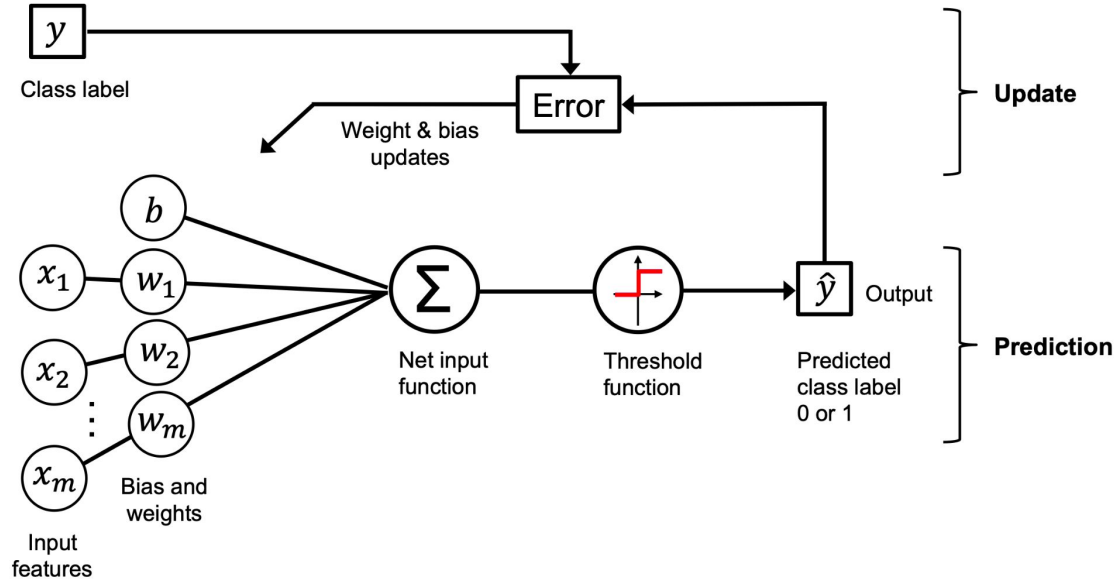


Figure 2.3: Examples of linearly and nonlinearly separable classes

Practice

Training a perceptron model on the Iris dataset

Implementing the Perceptron algorithm in Python.

Perceptron Algorithm

Adaptive linear neuron (Adaline)



Why Adaline Matters

- Introduces the concept of a **continuous loss function**
- Forms the basis for understanding:
 - **Logistic regression**
 - **Support Vector Machines**
 - **Multilayer neural networks**
 - **Linear regression**

How Adaline Learns (vs. Perceptron)

- **Adaline:**
 - Uses a **linear activation function** for weight updates
 - Compares the **true label** to the **continuous output** of the activation function
 - Computes the error and updates weights accordingly
 - Final prediction still uses a **threshold function** (like the perceptron)
- **Perceptron:**
 - Uses a **step function** to update weights
 - Compares the **true label** to the **binary predicted label** (0 or 1)
 - Only updates if prediction is wrong



How Adaline Learns (vs. Perceptron)

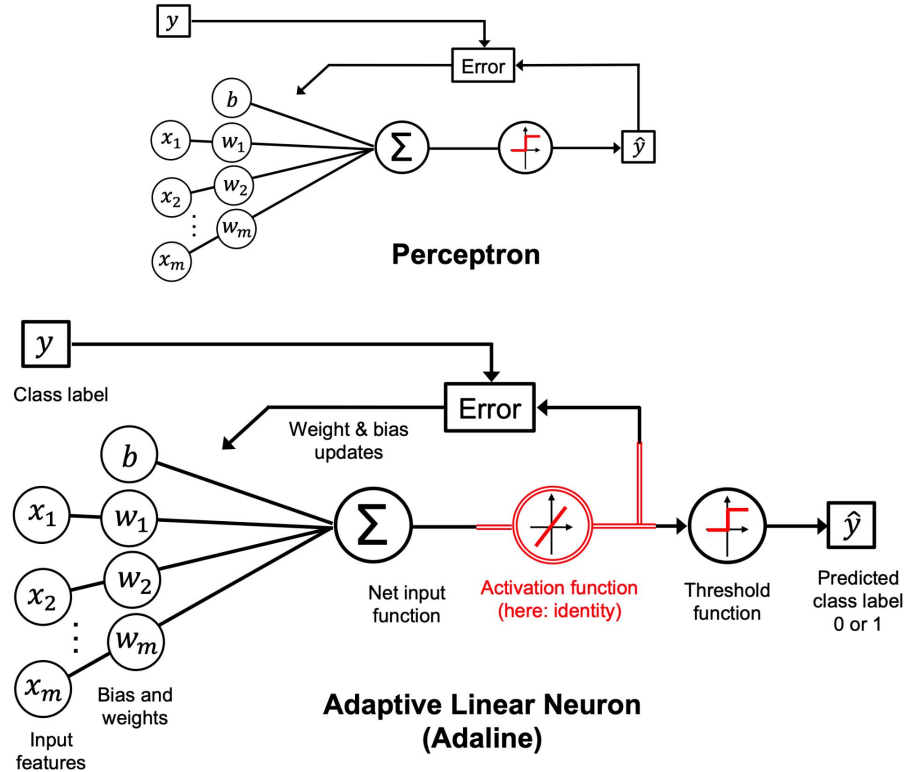


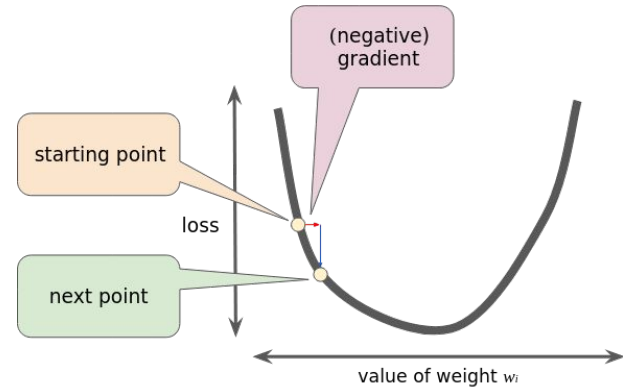
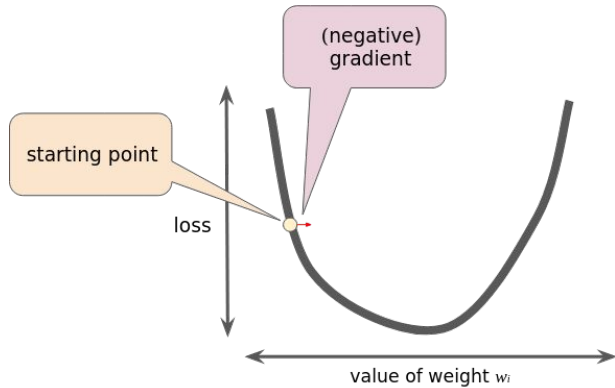
Figure 2.4: A comparison between a perceptron and the Adaline algorithm

Gradient Descent



Gradient Descent: Core Idea

- Think of it as **descending a hill** until a **local or global loss minimum** is reached.
- At each step, we move in the **opposite direction of the gradient** $\nabla L(\mathbf{w}, b)$
- The size of the step depends on:
 - The **learning rate** η
 - The **slope** of the loss surface (gradient)



Gradient Descent: Update Rules

To minimize $L(\mathbf{w}, b)$:

$$\frac{\partial L}{\partial w_j} = -\frac{2}{n} \sum_i (y^{(i)} - \sigma(z^{(i)})) x_j^{(i)}$$

$$\frac{\partial L}{\partial b} = -\frac{2}{n} \sum_i (y^{(i)} - \sigma(z^{(i)}))$$

$$\Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} L(\mathbf{w}, b), \quad \Delta b = -\eta \nabla_b L(\mathbf{w}, b)$$

$$\Delta w_j = -\eta \frac{\partial L}{\partial w_j} \quad \text{and} \quad \Delta b = -\eta \frac{\partial L}{\partial b}$$

Since we update all parameters simultaneously, our Adaline learning rule becomes:

$$\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}, \quad b := b + \Delta b$$

Learning Rate

Gradient descent algorithms multiply the gradient by a scalar known as the **learning rate** (also sometimes called **step size**) to determine the next point.

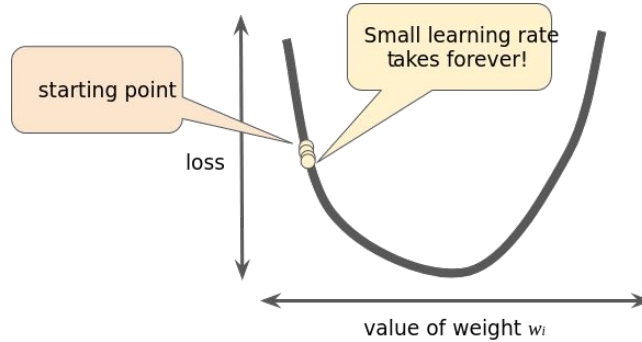
- For example, if the gradient magnitude is 2.5 and the learning rate is 0.01, then the gradient descent algorithm will pick the next point 0.025 away from the previous point.

Hyperparameters are the knobs that programmers tweak in machine learning algorithms.

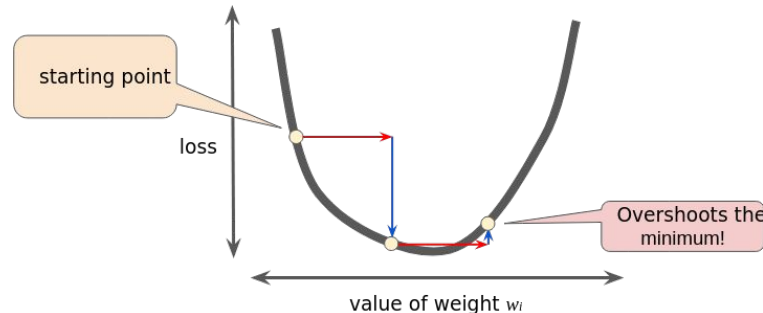
- Three common hyperparameters are:
 - **Learning rate**
 - **Batch size**
 - **Epochs**
- Most machine learning programmers spend a fair amount of time tuning the learning rate. If you pick a learning rate that is too small, learning will take too long:

Learning Rate

If you pick a learning rate that is too small, learning will take too long:



Conversely, if you specify a learning rate that is too large, the next point will perpetually bounce haphazardly across the bottom of the well



Learning Rate

An ideal learning rate

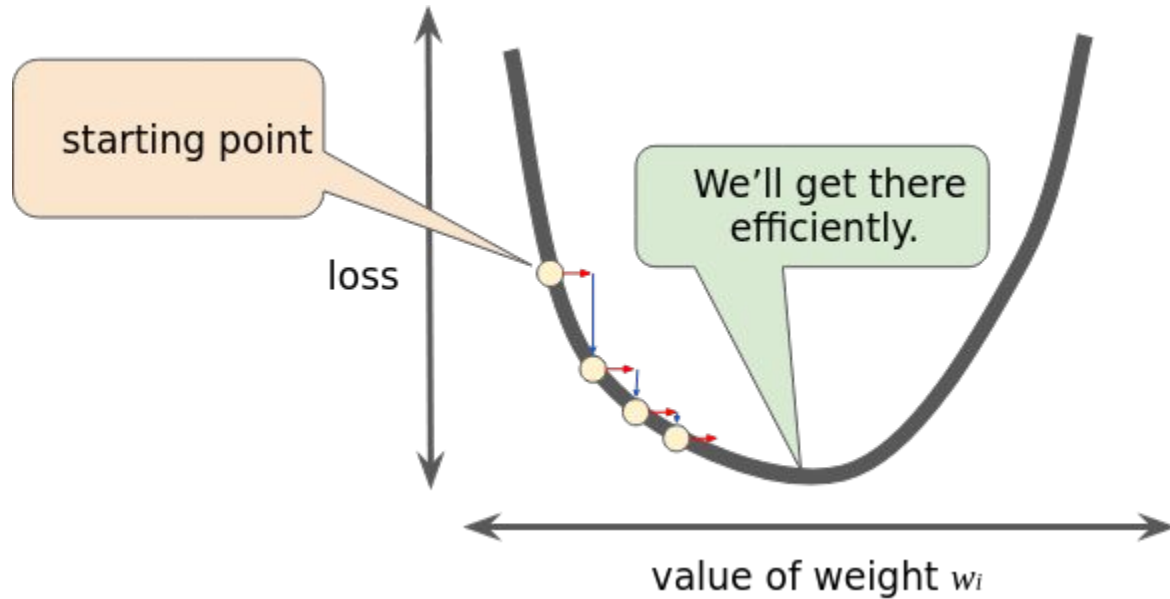


Figure 8: Learning rate is just right.

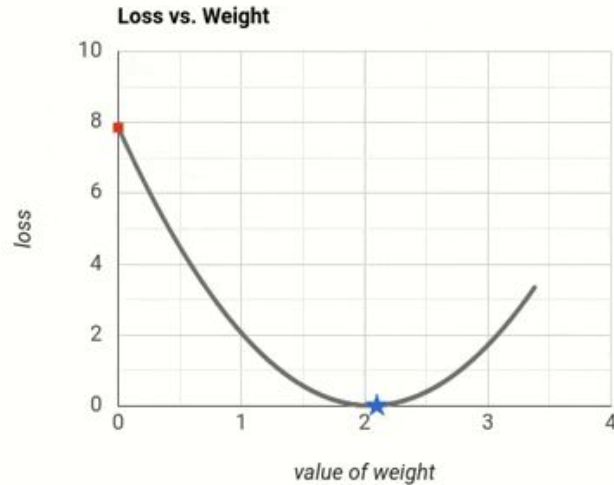
Optimizing Learning Rate

$lr = 0.03$

Set learning rate: 0.03

Execute single step: 0

Reset the graph:



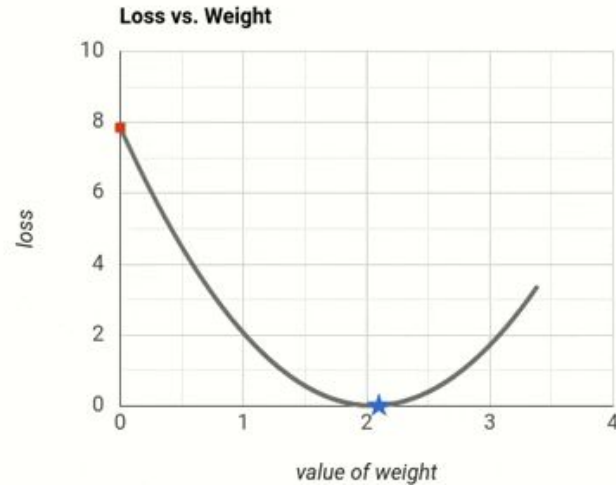
Optimizing Learning Rate

$lr = 0.1$

Set learning rate: 0.10

Execute single step: 0

Reset the graph:



Interactive Practice

How quickly does the model converges?

The learning rate hyperparameter.

Learning Rate



Saturdays.AI
Kigali

Improving Gradient Descent: Feature Scaling

🚀 Why Feature Scaling Matters

- Many ML algorithms, including **gradient descent**, perform better with **scaled features**
- Without scaling, features on different ranges can:
 - Slow convergence
 - Cause unstable or inefficient updates

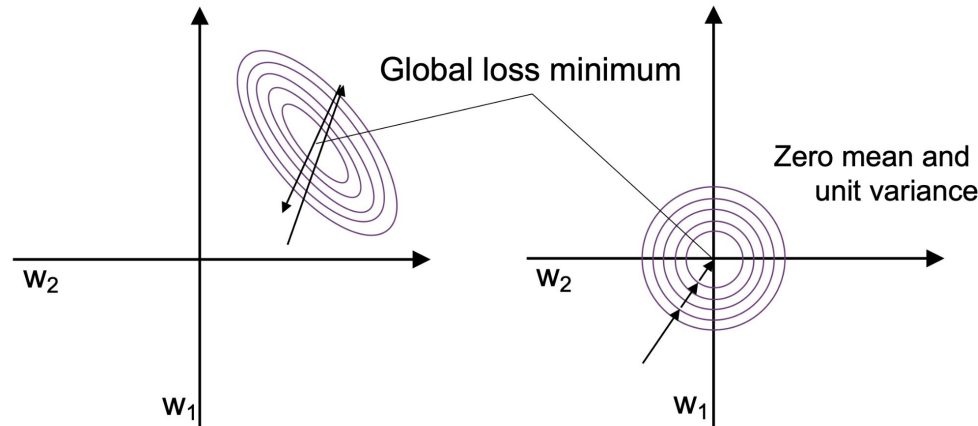


Figure 2.4: A comparison between a perceptron and the Adaline algorithm



Standardization: A Common Approach

- Shifts each feature to have:
 - **Mean = 0**
 - **Standard deviation = 1**
- Formula for the j-th feature:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

where μ_j is the mean and σ_j is the standard deviation

📌 Note: Standardization **does not** make the data normally distributed — it just centers and scales it.

How It Helps Gradient Descent

- Makes it easier to choose a **single learning rate** that works across all weights
- Prevents some weights from updating too slowly or too aggressively
- Leads to **faster, more stable convergence** toward the optimal solution

Large-Scale ML

In gradient descent, a **batch** is the total number of examples you use to calculate the gradient in a single iteration.

Full Batch Gradient Descent (Recap)

- Involves computing the **gradient over the entire training set**
- Each step toward the minimum uses **all data points**
- Also called batch **gradient descent**

The Challenge with Large Datasets

- Modern ML often involves **millions of examples**
- Running full-batch gradient descent:
 - Is **computationally expensive**
 - Requires a full pass through the dataset **for every update step**

Gradient Descent

Types

In gradient descent, a **batch** is the total number of examples you use to calculate the gradient in a single iteration.

- **Batch Gradient Descent:** Batch Size = Size of Training Set
- **Stochastic Gradient Descent (SDG):** Batch Size = 1
- **Mini-Batch Gradient Descent:** $1 < \text{Batch Size} < \text{Size of Training Set}$

Why Use SGD?

- **Faster convergence** due to more frequent updates
- Can **escape shallow local minima** in non-convex loss surfaces (thanks to noisy steps)

Practical Considerations

- **Shuffle training data** before each epoch to avoid cycles
- Present examples in **random order** for better generalization

Practice

Minimizing loss functions with gradient descent

Implementing Adaline in Python.

Adaline

Break



Saturdays.AI
Kigali

Practice

We give you code. What happens?

Run. Learn. Repeat.



Saturdays.AI
Kigali

Challenges & Next steps!

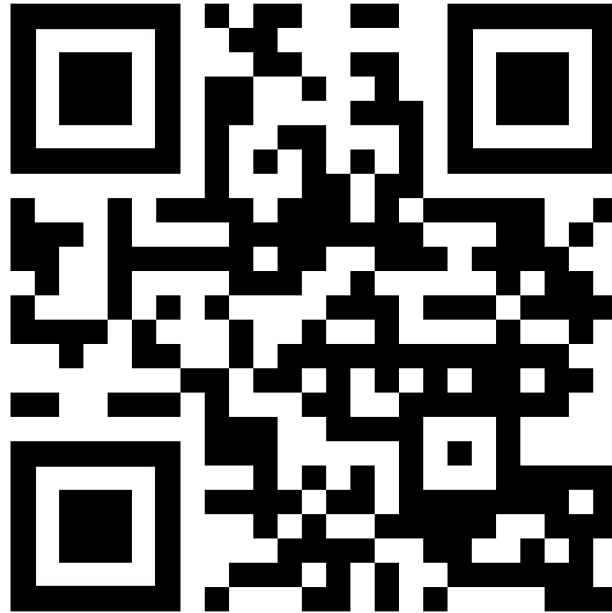


Saturdays.AI
Kigali

Best part of the day!



Kahoot!
kahoot.it



**Any
questions?**



Saturdays.AI
Kigali



Saturdays.AI
Kigali

THANKS



kigali@saturdays.ai



coming soon



coming soon